

A FROM-SCRATCH BUILD LOG & CODE WALKTHROUGH

Building a TCP Port Scanner in C++

Sockets, Threads, and select() — Understanding What Nmap
Is Really Doing, One Line of Code at a Time

SAM · PORTFOLIO PROJECT · C++17

Table of Contents

PART I — FOUNDATIONS

- 1 What This Project Is, and Why It Matters
- 2 Networking Fundamentals You Need First
- 3 Project Tour: Architecture at a Glance

PART II — BUILDING IT FROM SCRATCH

- 4 Step 0: Design Decisions Before Writing Any Code
- 5 Step 1: Project Skeleton and the Build System
- 6 Step 2: The Data Model — ScanOptions and PortResult
- 7 Step 3: Resolving the Target Host
- 8 Step 4: A Naive Connect Attempt (and Why It Fails)
- 9 Step 5: Non-Blocking Sockets and select()
- 10 Step 6: Scaling Out With a Thread Pool
- 11 Step 7: Grabbing Service Banners
- 12 Step 8: Guessing Service Names
- 13 Step 9: The Command-Line Interface (main.cpp)
- 14 Step 10: Compiling and Running It

PART III — THE CYBERSECURITY ANGLE

- 15 How This Maps to What Nmap Actually Does
- 16 Legal and Ethical Boundaries of Port Scanning

PART IV — APPENDIX

- 17 Full Source Code Listing
- 18 Where to Go Next

1 What This Project Is, and Why It Matters

A **port scanner** answers one simple question, over and over, very fast: "if I try to open a TCP connection to this address on this port, does anything answer?" That's it. Everything else — the threading, the timeouts, the banner grabbing — exists to ask that one question thousands of times without the whole program grinding to a halt.

This matters because it's exactly the first thing a security engineer, a network administrator, or an attacker does when they encounter an unfamiliar system: find out what's listening. Nmap, the industry-standard scanning tool, is built around this same core idea — `connect()` to a port and see what happens — plus a huge amount of additional engineering on top (raw-packet crafting, OS fingerprinting, a scripting engine, timing heuristics to avoid tripping intrusion-detection systems). This project builds the foundational layer that all of that sits on, so that using Nmap afterward feels like understanding a car engine instead of just knowing which pedal is the accelerator.

Why build this instead of just using Nmap?
Because typing `nmap target` teaches you nothing about *why* it works. Writing the socket calls yourself forces you to confront real operating-system behavior: what a non-blocking connect actually does, why a firewall makes a scanner hang instead of fail, why 65,535 sequential connection attempts would take hours. Those are systems-programming lessons, not scanning-tool trivia.

2 Networking Fundamentals You Need First

Before touching any code, it's worth being precise about the mechanics this project relies on.

Addresses and ports

An IPv4 address (like `192.168.1.10`) identifies a machine on a network. A **port** is a 16-bit number (0-65535) that identifies a specific service or process listening on that machine. A web server usually listens on port 80 (HTTP) or 443 (HTTPS); SSH usually listens on port 22. A "scan" of a host is really a sweep across some or all of these 65,536 possible port numbers.

The TCP three-way handshake

TCP connections are established with a three-step exchange:

Step	Direction	Packet	Meaning
1	Client to Server	SYN	"I would like to connect."
2	Server to Client	SYN-ACK	"Okay, go ahead." (port is open)
2 (alt)	Server to Client	RST	"No." (port is closed)
2 (alt)	—	(nothing)	Silence — a firewall is dropping the packet (port is filtered)
3	Client to Server	ACK	Handshake complete, connection established

A **connect scan** (what this project does, and what Nmap calls `-sT`) lets the operating system's TCP stack perform this entire handshake via the normal `connect()` system call. A **SYN scan** (Nmap's default, `-sS`) sends the raw SYN packet by hand and never completes step 3, which is faster and quieter but requires raw-socket privileges (root). Section 15 comes back to this distinction in more depth.

The sockets API, at a glance

Every network operation in this project goes through five POSIX calls:

Call	What it does
<code>socket()</code>	Creates an endpoint and returns a file descriptor (an integer handle) for it.
<code>connect()</code>	Initiates the TCP handshake toward a remote address/port.
<code>send()</code> / <code>recv()</code>	Write/read bytes on an established connection.
<code>select()</code>	Waits for one or more file descriptors to become ready (readable/writable), with a timeout.
<code>close()</code>	Releases the file descriptor and, for TCP, begins tearing down the connection.

3 Project Tour: Architecture at a Glance

The finished project has four files, deliberately kept small enough to read end-to-end in one sitting:

File	Responsibility
include/scanner.hpp	Public interface: the <code>PortScanner</code> class, the <code>ScanOptions</code> / <code>PortResult</code> data structures.
src/scanner.cpp	All the networking logic: DNS resolution, the non-blocking connect-with-timeout, the thread pool, banner grabbing.
src/main.cpp	Command-line parsing and human-readable output formatting. Knows nothing about sockets directly.
CMakeLists.txt	Build configuration.

The dependency direction is intentionally one-way: `main.cpp` depends on `scanner.hpp`, never the reverse. This mirrors the separation you'd want in any real tool — the "engine" (scanning logic) doesn't know or care whether it's being driven by a CLI, a test harness, or eventually a GUI.

Data flow, end to end
CLI arguments become a `ScanOptions` struct. `PortScanner::run()` resolves the host once, then spawns N worker threads. Each thread repeatedly claims the next port number and probes it. Open ports (with banners) accumulate into a shared, mutex-protected results vector. `main.cpp` sorts and prints them.

4 Step 0: Design Decisions Before Writing Any Code

Every real engineering project starts with a handful of decisions that shape everything downstream. Getting these right up front avoids painful rewrites later.

Decision	Choice made	Reasoning
Concurrency model	A fixed pool of OS threads	Simple to reason about, and C++'s <code><thread></code> is standard-library, no external dependency. An event loop (epoll/kqueue) would scale to more concurrent sockets with less memory, but adds real complexity for a learning project.
Work distribution	Single <code>std::atomic<int></code> counter	The work items are just consecutive integers (port numbers) — no need for a full producer/consumer queue when "give me the next number" is all any worker needs.
Connect strategy	Non-blocking socket + <code>select()</code>	A blocking <code>connect()</code> can hang for the OS's default TCP timeout (often 60+ seconds) on a filtered port. That's fatal for a scanner — timeouts must be under our control.
IP version	IPv4 only	Keeps the socket address structures and CLI simple for a first version; noted as a clear extension point later.
Language/standard	C++17	Gives us <code>std::atomic</code> , <code>std::thread</code> , structured code without needing Boost or any third-party networking library.

5 Step 1: Project Skeleton and the Build System

Start with the shape of the project before any logic exists:

DIRECTORY LAYOUT
tcp-port-scanner/\n├── CMakeLists.txt\n├── include/\n│ └── scanner.hpp\n├── src/\n│ ├── scanner.cpp\n│ └── main.cpp

CMake is used here instead of a raw Makefile because it handles compiler flags, the include path, and thread linking portably across Linux distributions without hand-written boilerplate. Here is the complete build file:

```
CMAKELISTS.TXT

cmake_minimum_required(VERSION 3.10)
project(tcp_port_scanner CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release)
endif()

add_compile_options(-Wall -Wextra)

find_package(Threads REQUIRED)

add_executable(scanner
    src/main.cpp
    src/scanner.cpp
)

target_include_directories(scanner PRIVATE include)
target_link_libraries(scanner PRIVATE Threads::Threads)
```

Walking through the non-obvious parts:

Line	Purpose
set(CMAKE_CXX_STANDARD 17)	Requires C++17 so <code>std::atomic</code> / <code>std::thread</code> features used later are guaranteed available.
add_compile_options(-Wall -Wextra)	Turns on the compiler warnings that catch real bugs — an uninitialized struct member, a signed/unsigned comparison, a shadowed variable.
find_package(Threads REQUIRED)	Locates the platform's threading library (pthreads on Linux) so <code>std::thread</code> actually links.
target_link_libraries(scanner PRIVATE Threads::Threads)	Links against it. Without this, the binary compiles but fails at link time with undefined references to pthread symbols.

6 Step 2: The Data Model — ScanOptions and PortResult

Before writing any networking code, define what a "scan" and a "result" actually are. Getting the data model right first makes the rest of the implementation almost mechanical.

```
INCLUDE/SCANNER.HPP (EXCERPT)

struct PortResult {
    int port = 0;
    bool open = false;
    std::string service; // best-guess service name
    std::string banner;  // text grabbed from the socket, if any
};

struct ScanOptions {
    std::string host;
    int startPort = 1;
    int endPort = 1024;
    int numThreads = 100;
    int connectTimeoutMs = 500;
    int bannerTimeoutMs = 500;
    bool grabBanners = true;
};
```

`ScanOptions` is a plain bag of configuration — everything the CLI layer needs to specify and nothing the scanning engine needs to guess. `PortResult` is deliberately narrow: just enough fields to answer "what did we find," which keeps `main.cpp`'s printing logic trivial later.

The `PortScanner` class itself is declared around these two types:

INCLUDE/SCANNER.HPP (EXCERPT)

```
class PortScanner {
public:
    explicit PortScanner(ScanOptions options);
    bool run(); // resolves host, scans, returns false on DNS failure
    std::vector<PortResult> openPorts() const;
    std::chrono::milliseconds elapsed() const { return elapsed_; }
    const std::string& resolvedIp() const { return resolvedIp_; }

private:
    void workerLoop();
    PortResult probePort(int port);
    std::string tryGrabBanner(int sockFd, int port);

    ScanOptions opts_;
    std::string resolvedIp_;
    std::atomic<int> nextPort_{0};
    std::mutex resultsMutex_;
    std::vector<PortResult> openResults_;
    std::chrono::milliseconds elapsed_{0};
};
```

Design note: public vs. private

Notice that `probePort`, `workerLoop`, and `tryGrabBanner` are all private — they are implementation details of "how a scan happens." The public surface is deliberately tiny: construct with options, call `run()`, read results. That's the entire contract anything outside this class needs to know.

7 Step 3: Resolving the Target Host

The user might type an IP literal (`192.168.1.10`) or a hostname (`scanme.nmap.org`). Both need to become a concrete IPv4 address before any socket code can run. The POSIX function for this is `getaddrinfo()`:

SRC/SCANNER.CPP (EXCERPT)

```
std::string resolveHost(const std::string& host) {
    struct addrinfo hints{};
    hints.ai_family = AF_INET; // IPv4 only, for this project
    hints.ai_socktype = SOCK_STREAM; // TCP

    struct addrinfo* result = nullptr;
    int rc = getaddrinfo(host.c_str(), nullptr, &hints, &result);
    if (rc != 0 || result == nullptr) {
        return ""; // resolution failed -- bad hostname or no DNS
    }

    char ipStr[INET_ADDRSTRLEN] = {0};
    auto* addr = reinterpret_cast<struct sockaddr_in*>(result->ai_addr);
    inet_ntop(AF_INET, &addr->sin_addr, ipStr, sizeof(ipStr));
    freeaddrinfo(result); // getaddrinfo allocates -- always free it
    return std::string(ipStr);
}
```

Two details worth internalizing:

- `getaddrinfo` allocates a linked list of results (a host can have multiple addresses); `freeaddrinfo()` must always be called to avoid a leak.
- Resolution happens **once**, before scanning starts — not once per port. Repeating a DNS lookup thousands of times would be wasteful and would make timing behavior unpredictable if the DNS server itself is slow.

8 Step 4: A Naive Connect Attempt (and Why It Fails)

It helps to deliberately write the wrong version first, then feel exactly why it's wrong. Here is the simplest possible "is this port open" check:

```
NAIVE_CONNECT.CPP -- DO NOT SHIP THIS
```

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

bool isOpenNaive(const std::string& ip, int port) {
    int sockFd = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    inet_pton(AF_INET, ip.c_str(), &addr.sin_addr);

    // PROBLEM: if the port is filtered (firewall silently drops the
    // SYN packet), this call can block for the OS's default TCP
    // connect timeout - often 60+ seconds, PER PORT.
    int rc = connect(sockFd, (struct sockaddr*)&addr, sizeof(addr));

    close(sockFd);
    return rc == 0;
}
```

Why this breaks in practice

Sockets are blocking by default. If the target port is **closed**, the OS usually gets a TCP RST back almost instantly and `connect()` returns quickly with an error. But if the port is **filtered** — a firewall silently drops the SYN with no reply at all — the kernel has no way to know that. It just keeps waiting, retransmitting the SYN internally, until its own connection-attempt timeout expires. On Linux that default is often **around a minute**. Multiply that by a few dozen filtered ports in a 1-65535 sweep and a "quick scan" becomes an overnight job.

This is the single most important lesson in the whole project: **the network doesn't guarantee a timely answer, so the program must enforce its own timeout**. That's what Step 5 builds.

9 Step 5: Non-Blocking Sockets and `select()`

The fix has three parts: make the socket non-blocking, kick off the connect so it returns immediately, then use `select()` to wait — with a timeout we choose — for the handshake to actually finish.

5a. Make the socket non-blocking

```
int sockFd = socket(AF_INET, SOCK_STREAM, 0);

// Ask the kernel to make every I/O call on this socket non-blocking.
int flags = fcntl(sockFd, F_GETFL, 0);
fcntl(sockFd, F_SETFL, flags | O_NONBLOCK);
```

`fcntl(fd, F_GETFL, 0)` reads the socket's current flags; OR-ing in `O_NONBLOCK` and writing it back with `F_SETFL` tells the kernel: "never make me wait on this file descriptor — if an operation can't complete immediately, fail it right away instead."

5b. Kick off `connect()` and interpret its immediate return

```
int rc = connect(sockFd, (struct sockaddr*)&addr, sizeof(addr));

if (rc < 0 && errno != EINPROGRESS) {
    // Something failed immediately and for real (e.g. network unreachable).
    close(sockFd);
    return -1;
}

if (rc == 0) {
    // Extremely rare for TCP, but possible for something like a
    // loopback connection that completes synchronously.
    return sockFd;
}

// rc == -1 and errno == EINPROGRESS: this is the expected, normal case.
// The three-way handshake is happening in the background. We now need
// to find out, without blocking, when it finishes (or times out).
```

On a non-blocking socket, `connect()` essentially always returns `-1` with `errno == EINPROGRESS` — "started, not finished yet." That's not an error condition to handle defensively; it's the expected, normal path this whole design is built around.

5c. Wait for completion with `select()` -- this is the timeout

```

fd_set writeSet;
FD_ZERO(&writeSet);
FD_SET(sockFd, &writeSet);

struct timeval tv{};
tv.tv_sec = timeoutMs / 1000;
tv.tv_usec = (timeoutMs % 1000) * 1000;

// select() blocks until sockFd becomes writable (handshake finished,
// one way or another) OR the timeout expires — whichever comes first.
int rc = select(sockFd + 1, nullptr, &writeSet, nullptr, &tv);

if (rc <= 0) {
    // rc == 0 -> timed out, nothing happened in time (port is likely
    // filtered / firewalled — no response at all)
    // rc < 0 -> select() itself failed
    close(sockFd);
    return -1;
}

```

This is the crux of the entire timeout mechanism. `select()` takes a set of file descriptors to watch and a `struct timeval` deadline, and blocks *only until either something happens or the deadline passes* — never longer. A filtered port that would have hung for 60 seconds under the naive approach now costs exactly `connectTimeoutMs`, because we told the kernel to stop waiting at that point.

5d. Writable doesn't mean successful -- check `SO_ERROR`

```

// The socket became "writable" — but that only means the connection
// attempt is FINISHED, not that it SUCCEEDED. A refused connection
// (RST received) also makes the socket writable. We have to ask the
// kernel what actually happened using SO_ERROR.
int sockErr = 0;
socklen_t len = sizeof(sockErr);
getsockopt(sockFd, SOL_SOCKET, SO_ERROR, &sockErr, &len);

if (sockErr != 0) {
    // Connection was refused, unreachable, etc. Port is closed.
    close(sockFd);
    return -1;
}

// sockErr == 0: the TCP handshake genuinely completed. Port is open.

```

This is a subtle POSIX gotcha worth calling out explicitly: a socket becomes "writable" both when a connection *succeeds* and when it's *refused*. `select()` alone can't tell those apart. The only reliable way is asking the kernel directly via `getsockopt(..., SO_ERROR, ...)`, which returns the pending error code (zero means "no error — genuinely connected").

Putting it together

The real `connectWithTimeout()` function in `scanner.cpp` is exactly these four steps chained: create + set non-blocking → call `connect()` → `select()` with a deadline → verify with `SO_ERROR`. See the full listing in Section 17 for the version with error handling for every intermediate failure path.

10 Step 6: Scaling Out With a Thread Pool

Even with a bounded timeout, scanning is still fundamentally latency-bound: each connection attempt takes some milliseconds of round-trip time, and doing them one after another wastes almost all of that time waiting.

BACK-OF-ENVELOPE MATH

```

// 1000 ports, ~1ms average round-trip on a LAN:
// 1000 x 1ms = ~1 second -- fine
//
// But add just a handful of filtered ports at a 500ms timeout each:
// 10 filtered ports x 500ms = 5 extra seconds
// 65535 ports, worst case, sequential, 500ms timeout each
// = up to 9 HOURS if the host silently drops most packets
//
// Scanning ports one at a time is the bottleneck, not the network.

```

The fix is to run many connection attempts concurrently. This project uses a fixed pool of worker threads that all pull from one shared atomic counter representing "the next port nobody has claimed yet":

```

SRC/SCANNER.CPP -- WORKERLOOP()

std::atomic<int> nextPort_{0};

// Every worker thread runs this same loop. There is no shared queue,
// no lock to take just to find out "what's next" – each thread simply
// increments one atomic counter and claims whatever value it got back.
// fetch_add() is atomic: two threads can never receive the same port.
void PortScanner::workerLoop() {
    while (true) {
        int port = nextPort_.fetch_add(1);
        if (port > opts_.endPort) {
            return; // no ports left – this worker is done
        }

        PortResult result = probePort(port);
        if (result.open) {
            std::lock_guard<std::mutex> lock(resultsMutex_);
            openResults_.push_back(std::move(result));
        }
    }
}

```

`fetch_add(1)` is an atomic read-modify-write: it returns the counter's current value and increments it, as one indivisible operation. Two threads calling it "simultaneously" are still serialized by the CPU/kernel at the hardware level, so no two workers can ever be handed the same port number — without a single explicit lock for this part.

The results vector, on the other hand, is genuinely shared mutable state that multiple threads write to, so it needs a real mutex:

```

if (result.open) {
    std::lock_guard<std::mutex> lock(resultsMutex_);
    openResults_.push_back(std::move(result));
}

```

Note how small the critical section is — the lock is held only for the vector push, not for the network probe itself. Keeping locked regions short is what keeps a multithreaded scanner from becoming, accidentally, a slow single-threaded one.

Spawning and joining the pool in `run()`:

```

SRC/SCANNER.CPP -- RUN()

nextPort_.store(opts_.startPort);

std::vector<std::thread> workers;
for (int i = 0; i < threadCount; ++i) {
    workers.emplace_back(&PortScanner::workerLoop, this);
}
for (auto& t : workers) {
    t.join(); // wait for every worker to finish before returning results
}

```

Why this scales

With 100 threads each waiting on their own independent socket, 100 connection attempts are effectively in flight at once. A scan that would take minutes sequentially against a mostly-filtered host completes in roughly `totalPorts / numThreads` round trips instead of `totalPorts` of them.

11 Step 7: Grabbing Service Banners

Knowing a port is open is useful; knowing *what's running on it* is more useful. Many services announce their identity the moment a TCP connection completes — SSH sends a version string immediately, FTP and SMTP servers send a greeting banner. HTTP servers are the opposite: they wait silently for the client to send a request first.

```

SRC/SCANNER.CPP -- TRYGRABBANNER()

const std::string service = wellKnownService(port);

if (service == "http" || service == "http-alt") {
    // HTTP servers wait for the client to speak first.
    // Sending a minimal, harmless HEAD request is enough to make
    // almost any web server reply with its status line and headers.
    static const char* kProbe = "HEAD / HTTP/1.0\r\n\r\n";
    send(sockFd, kProbe, std::strlen(kProbe), 0);
}
// SSH, FTP, SMTP, and many others announce themselves the instant
// the TCP handshake completes -- no probe needed, just recv().

```

For everything else, the scanner just tries to read whatever the far end sends on its own — but that read still needs a timeout, for the same reason connecting did:

```

struct timeval tv{};
tv.tv_sec  = opts_.bannerTimeoutMs / 1000;
tv.tv_usec = (opts_.bannerTimeoutMs % 1000) * 1000;

// SO_RCVTIMEO caps how long recv() is allowed to block waiting for
// data. Without this, a port that accepts connections but never
// sends anything would hang the worker thread forever.
setsockopt(sockFd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

char buf[512] = {0};
ssize_t n = recv(sockFd, buf, sizeof(buf) - 1, 0);

```

`SO_RCVTIMEO` is the receive-side equivalent of the connect timeout from Step 5: it bounds how long a single `recv()` call is allowed to block. Without it, an open port that accepts connections but never sends a byte (common for custom TCP services) would freeze that worker thread indefinitely.

The raw bytes that come back are then cleaned up for display — carriage returns and newlines collapsed to spaces, trailing whitespace trimmed — so a multi-line banner prints as one tidy line in the CLI output table.

12 Step 8: Guessing Service Names

Port numbers alone (22, 443) aren't very readable. A small static lookup table — essentially a hand-rolled miniature of `/etc/services` — maps the common ones to human-readable names:

```

SRC/SCANNER.CPP

static const std::unordered_map<int, std::string> kServices = {
    {22, "ssh"}, {80, "http"}, {443, "https"}, {3306, "mysql"},
    // ... trimmed for the walkthrough, full table in scanner.cpp
};

std::string wellKnownService(int port) {
    auto it = kServices.find(port);
    return it != kServices.end() ? it->second : "unknown";
}

```

This is intentionally a best-effort guess, not a guarantee — nothing stops an administrator from running SSH on port 8022 instead of 22. That's precisely why banner grabbing (Step 7) matters as a second, more reliable signal: the banner text itself often names the exact software and version, regardless of which port it's bound to.

13 Step 9: The Command-Line Interface (main.cpp)

`main.cpp` is deliberately "dumb" — it parses arguments, builds a `ScanOptions`, calls `PortScanner::run()`, and formats the result. It contains no socket code at all.

```

SRC/MAIN.CPP -- ARGUMENT PARSING

for (int i = 1; i < argc; ++i) {
    std::string arg = argv[i];
    if (arg == "-h" || arg == "--help") {
        printUsage(argv[0]);
        return 0;
    } else if (arg == "--no-banner") {
        opts.grabBanners = false;
    } else if (arg == "-t" && i + 1 < argc) {
        opts.numThreads = std::atoi(argv[++i]); // consume the next token too
    } else if (arg == "-T" && i + 1 < argc) {
        opts.connectTimeoutMs = std::atoi(argv[++i]);
    } else if (!arg.empty() && arg[0] == '-') {
        std::cerr << "Unknown option: " << arg << "\n";
        return 1;
    } else {
        positional.push_back(arg); // host, start_port, end_port land here
    }
}

```

Two things about this loop are worth noting as general C++ CLI-parsing patterns:

- Options that take a value (`-t`, `-T`) advance the loop index an extra time with `argv[++i]` to consume their argument — a common source of off-by-one bugs if forgotten.
- Everything that isn't a recognized flag falls through to `positional`, which is later unpacked as `host`, `start_port`, `end_port` in that order. This keeps the parser simple without pulling in a CLI-parsing library for a project this size.

After parsing, `main.cpp` validates the inputs before ever touching the network — port range sanity, thread count sanity — then prints results as a simple tab-separated table sorted by port number ascending (sorting happens once, inside `PortScanner::run()`, right after all threads join).

14 Step 10: Compiling and Running It

With all the pieces in place, the build and a real run look like this:

```

TERMINAL
$ mkdir build && cd build
$ cmake .. -DCMAKE_BUILD_TYPE=Release
$ make

$ ./scanner 127.0.0.1 1 1024
Scanning 127.0.0.1 ports 1-1024 with 100 threads...
Resolved to 127.0.0.1

PORT    STATE    SERVICE    BANNER
22      open     ssh        SSH-2.0-OpenSSH_9.6

Scanned 1024 ports in 46 ms. 1 open.
```

A few things to verify when you build this yourself:

- `-Wall -Wextra` should produce **zero** warnings — if it doesn't, treat every warning as a real bug to fix, not noise to ignore.
- Try scanning a range that includes both an open port (e.g. an SSH server) and a large block of closed ports, and time it — then try again with `-t 1` (single-threaded) to feel directly how much the thread pool is buying you.
- Run once with `--no-banner` and once without, on a target you control, to see the banner-grabbing time cost versus the information gained.

15 How This Maps to What Nmap Actually Does

This project implements exactly one of Nmap's many scan techniques. Seeing it in context clarifies both what was built and what was deliberately left out.

Technique	Nmap flag	Mechanism	Privilege needed
Connect scan	<code>-sT</code>	Full TCP handshake via <code>connect()</code> — what this project does	None (normal user)
SYN scan	<code>-sS</code>	Hand-crafted raw SYN packet; connection never completes (handshake abandoned after SYN-ACK)	Root / raw sockets
UDP scan	<code>-sU</code>	Sends UDP packets, infers state from ICMP responses or silence	Root / raw sockets
Version detection	<code>-sV</code>	A much larger, protocol-specific probe database than a single HTTP HEAD request	None

The reason SYN scans need root is that constructing a raw TCP packet by hand (rather than asking the kernel's TCP stack to do it via `connect()`) requires a raw socket (`SOCK_RAW`), which the OS restricts to privileged processes — a scanner forging packet headers is indistinguishable, from the kernel's point of view, from a much wider class of low-level network tools that could otherwise be misused.

SYN scans are also stealthier and faster: because the handshake is never completed, many logging systems that only record *established* connections never see the probe at all, and there's no full connection teardown overhead per port. Building a SYN scanner is a natural next step after this project (noted again in Section 18) — it exercises raw sockets and manual IP/TCP header construction, a meaningfully different skill from the standard sockets API used here.

16 Legal and Ethical Boundaries of Port Scanning

Port scanning itself is a normal, everyday part of network administration and security work — but running it against systems you don't own or don't have explicit permission to test can violate computer-misuse laws in many countries, even when no damage is done and no data is accessed.

Practical rule of thumb

Only scan hosts you own, hosts on a network you control (like your own home lab or a VM), or hosts that explicitly permit it — for example, `scanme.nmap.org` is maintained by the Nmap project specifically as a legal public target for testing scanners. When in doubt, get written permission first.

This isn't a limitation of the tool — it's the same standard that applies to Nmap, Masscan, or any other scanner. The code teaches the mechanics; using it responsibly is a separate, non-negotiable professional habit.

The complete, unabridged source for the project as built and tested.

```
include/scanner.hpp
```

```
#pragma once

#include <string>
#include <vector>
#include <mutex>
#include <atomic>
#include <chrono>

namespace pscan {

// Result of probing a single port.
struct PortResult {
    int port = 0;
    bool open = false;
    std::string service; // best-guess service name (from /etc/services style lookup)
    std::string banner; // text grabbed from the socket, if any
};

struct ScanOptions {
    std::string host;
    int startPort = 1;
    int endPort = 1024;
    int numThreads = 100; // concurrent workers
    int connectTimeoutMs = 500; // per-connection timeout
    int bannerTimeoutMs = 500; // how long to wait for a banner
    bool grabBanners = true;
};

class PortScanner {
public:
    explicit PortScanner(ScanOptions options);

    // Resolves the host, then scans [startPort, endPort] using numThreads workers.
    // Returns false if the host could not be resolved.
    bool run();

    // Open ports found, sorted ascending. Only valid after run() returns.
    std::vector<PortResult> openPorts() const;

    // Total elapsed wall-clock time for the last run().
    std::chrono::milliseconds elapsed() const { return elapsed_; }

    const std::string& resolvedIp() const { return resolvedIp_; }

private:
    void workerLoop();
    PortResult probePort(int port);
    std::string tryGrabBanner(int sockFd, int port);

    ScanOptions opts_;
    std::string resolvedIp_;

    std::atomic<int> nextPort_{0};
    std::mutex resultsMutex_;
    std::vector<PortResult> openResults_;

    std::chrono::milliseconds elapsed_{0};
};

// Best-effort service name for a well-known port (e.g. 80 -> "http").
// Falls back to "unknown" if not recognized.
std::string wellKnownService(int port);

} // namespace pscan
```

```
src/scanner.cpp
```

```
#include "scanner.hpp"

#include <algorithm>
#include <cstring>
#include <iostream>
#include <thread>
#include <unordered_map>
#include <vector>

#include <arpa/inet.h>
#include <fcntl.h>
#include <netdb.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>

namespace pscan {
```

```

namespace {

// Resolve a hostname or IP literal to a dotted-quad string.
// Returns empty string on failure.
std::string resolveHost(const std::string& host) {
    struct addrinfo hints{};
    hints.ai_family = AF_INET; // IPv4 keeps the socket code simple; could extend to AF_UNSPEC
    hints.ai_socktype = SOCK_STREAM;

    struct addrinfo* result = nullptr;
    int rc = getaddrinfo(host.c_str(), nullptr, &hints, &result);
    if (rc != 0 || result == nullptr) {
        return "";
    }

    char ipStr[INET_ADDRSTRLEN] = {};
    auto* addr = reinterpret_cast<struct sockaddr_in*>(result->ai_addr);
    inet_ntop(AF_INET, &addr->sin_addr, ipStr, sizeof(ipStr));
    freeaddrinfo(result);
    return std::string(ipStr);
}

// Attempts a TCP connect with an explicit timeout using a non-blocking socket + select().
// Returns the connected socket fd on success (caller owns it), or -1 on failure/timeout.
int connectWithTimeout(const std::string& ip, int port, int timeoutMs) {
    int sockFd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockFd < 0) {
        return -1;
    }

    // Make the socket non-blocking so connect() returns immediately.
    int flags = fcntl(sockFd, F_GETFL, 0);
    fcntl(sockFd, F_SETFL, flags | O_NONBLOCK);

    struct sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(static_cast<uint16_t>(port));
    inet_pton(AF_INET, ip.c_str(), &addr.sin_addr);

    int rc = connect(sockFd, reinterpret_cast<struct sockaddr*>(&addr), sizeof(addr));
    if (rc < 0 && errno != EINPROGRESS) {
        close(sockFd);
        return -1;
    }

    if (rc == 0) {
        // Connected immediately (rare, e.g. localhost).
        fcntl(sockFd, F_SETFL, flags); // restore blocking mode for later I/O
        return sockFd;
    }

    // Wait for the socket to become writable, which signals connect() completion.
    fd_set writeSet;
    FD_ZERO(&writeSet);
    FD_SET(sockFd, &writeSet);

    struct timeval tv{};
    tv.tv_sec = timeoutMs / 1000;
    tv.tv_usec = (timeoutMs % 1000) * 1000;

    rc = select(sockFd + 1, nullptr, &writeSet, nullptr, &tv);
    if (rc <= 0) {
        // Timed out, or select() itself failed.
        close(sockFd);
        return -1;
    }

    // Writable doesn't guarantee success -- check SO_ERROR to be sure.
    int sockErr = 0;
    socklen_t len = sizeof(sockErr);
    getsockopt(sockFd, SOL_SOCKET, SO_ERROR, &sockErr, &len);
    if (sockErr != 0) {
        close(sockFd);
        return -1;
    }

    fcntl(sockFd, F_SETFL, flags); // restore blocking mode for recv/send with its own timeout
    return sockFd;
}

} // namespace

std::string wellKnownService(int port) {
    static const std::unordered_map<int, std::string> kServices = {
        {21, "ftp"}, {22, "ssh"}, {23, "telnet"}, {25, "smtp"}, {53, "dns"},
        {80, "http"}, {110, "pop3"}, {111, "rpcbind"}, {135, "msrpc"},
        {139, "netbios-ssn"}, {143, "imap"}, {389, "ldap"}, {443, "https"},
        {445, "microsoft-ds"}, {587, "smtp-submission"}, {993, "imaps"},
        {995, "pop3s"}, {1433, "mssql"}, {1521, "oracle"}, {3306, "mysql"},
        {3389, "rdp"}, {5432, "postgresql"}, {5900, "vnc"}, {6379, "redis"},
        {8080, "http-alt"}, {8443, "https-alt"}, {27017, "mongodb"},
    };
    auto it = kServices.find(port);
    return it != kServices.end() ? it->second : "unknown";
}

PortScanner::PortScanner(ScanOptions options) : opts_(std::move(options)) {}

std::string PortScanner::tryGrabBanner(int sockFd, int port) {
    // Some services (HTTP-family) stay silent until spoken to; others
    // (SSH, FTP, SMTP) announce themselves immediately on connect.

```

```

const std::string service = wellKnownService(port);
if (service == "http" || service == "http-alt") {
    static const char* kProbe = "HEAD / HTTP/1.0\r\n\r\n";
    send(sockFd, kProbe, std::strlen(kProbe), 0);
}

struct timeval tv{};
tv.tv_sec = opts_.bannerTimeoutMs / 1000;
tv.tv_usec = (opts_.bannerTimeoutMs % 1000) * 1000;
setsockopt(sockFd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

char buf[512] = {0};
ssize_t n = recv(sockFd, buf, sizeof(buf) - 1, 0);
if (n <= 0) {
    return "";
}

std::string banner(buf, static_cast<size_t>(n));
// Collapse the banner to a single readable line for clean CLI output.
std::replace(banner.begin(), banner.end(), '\r', ' ');
std::replace(banner.begin(), banner.end(), '\n', ' ');
// Trim trailing whitespace.
while (!banner.empty() && banner.back() == ' ') banner.pop_back();
return banner;
}

PortResult PortScanner::probePort(int port) {
    PortResult result;
    result.port = port;
    result.service = wellKnownService(port);

    int sockFd = connectWithTimeout(resolvedIp_, port, opts_.connectTimeoutMs);
    if (sockFd < 0) {
        result.open = false;
        return result;
    }

    result.open = true;
    if (opts_.grabBanners) {
        result.banner = tryGrabBanner(sockFd, port);
    }
    close(sockFd);
    return result;
}

void PortScanner::workerLoop() {
    while (true) {
        int port = nextPort_.fetch_add(1);
        if (port > opts_.endPort) {
            return;
        }

        PortResult result = probePort(port);
        if (result.open) {
            std::lock_guard<std::mutex> lock(resultsMutex_);
            openResults_.push_back(std::move(result));
        }
    }
}

bool PortScanner::run() {
    resolvedIp_ = resolveHost(opts_.host);
    if (resolvedIp_.empty()) {
        return false;
    }

    nextPort_.store(opts_.startPort);
    openResults_.clear();

    const int totalPorts = opts_.endPort - opts_.startPort + 1;
    const int threadCount = std::max(1, std::min(opts_.numThreads, totalPorts));

    auto startTime = std::chrono::steady_clock::now();

    std::vector<std::thread> workers;
    workers.reserve(static_cast<size_t>(threadCount));
    for (int i = 0; i < threadCount; ++i) {
        workers.emplace_back(&PortScanner::workerLoop, this);
    }
    for (auto& t : workers) {
        t.join();
    }

    elapsed_ = std::chrono::duration_cast<std::chrono::milliseconds>(
        std::chrono::steady_clock::now() - startTime);

    std::sort(openResults_.begin(), openResults_.end(),
        [](const PortResult& a, const PortResult& b) { return a.port < b.port; });

    return true;
}

std::vector<PortResult> PortScanner::openPorts() const {
    return openResults_;
}

} // namespace pscan

```

```

#include "scanner.hpp"

#include <cstdlib>
#include <iostream>
#include <string>
#include <vector>

namespace {

void printUsage(const char* progName) {
    std::cerr <<
        "Usage: " << progName << " <host> [start_port] [end_port] [options]\n"
        "Positional arguments:\n"
        "  host      Hostname or IPv4 address to scan\n"
        "  start_port First port in range (default: 1)\n"
        "  end_port   Last port in range (default: 1024)\n\n"
        "Options:\n"
        "  -t <n>      Number of concurrent threads (default: 100)\n"
        "  -T <ms>     Per-port connect timeout in milliseconds (default: 500)\n"
        "  --no-banner Skip banner grabbing (faster scans)\n"
        "  -h, --help  Show this help message\n"
        "Examples:\n"
        "  " << progName << " 192.168.1.10 1 1000\n"
        "  " << progName << " scanme.nmap.org 1 65535 -t 200 -T 300\n";
}

} // namespace

int main(int argc, char* argv[]) {
    if (argc < 2) {
        printUsage(argv[0]);
        return 1;
    }

    pscan::ScanOptions opts;

    std::vector<std::string> positional;
    for (int i = 1; i < argc; ++i) {
        std::string arg = argv[i];
        if (arg == "-h" || arg == "--help") {
            printUsage(argv[0]);
            return 0;
        } else if (arg == "--no-banner") {
            opts.grabBanners = false;
        } else if (arg == "-t" && i + 1 < argc) {
            opts.numThreads = std::atoi(argv[++i]);
        } else if (arg == "-T" && i + 1 < argc) {
            opts.connectTimeoutMs = std::atoi(argv[++i]);
        } else if (!arg.empty() && arg[0] == '-') {
            std::cerr << "Unknown option: " << arg << "\n";
            printUsage(argv[0]);
            return 1;
        } else {
            positional.push_back(arg);
        }
    }

    if (positional.empty()) {
        printUsage(argv[0]);
        return 1;
    }

    opts.host = positional[0];
    if (positional.size() >= 2) opts.startPort = std::atoi(positional[1].c_str());
    if (positional.size() >= 3) opts.endPort = std::atoi(positional[2].c_str());

    if (opts.startPort < 1 || opts.endPort > 65535 || opts.startPort > opts.endPort) {
        std::cerr << "Error: invalid port range " << opts.startPort << "-" << opts.endPort << "\n";
        return 1;
    }

    if (opts.numThreads < 1) {
        std::cerr << "Error: thread count must be at least 1\n";
        return 1;
    }

    std::cout << "Scanning " << opts.host << " ports " << opts.startPort
        << "-" << opts.endPort << " with " << opts.numThreads << " threads...\n";

    pscan::PortScanner scanner(opts);
    if (!scanner.run()) {
        std::cerr << "Error: could not resolve host '" << opts.host << "'\n";
        return 1;
    }

    std::cout << "Resolved to " << scanner.resolvedIp() << "\n\n";

    auto results = scanner.openPorts();
    if (results.empty()) {
        std::cout << "No open ports found in range.\n";
    } else {
        std::cout << "PORT\tSTATE\tSERVICE\t\tBANNER\n";
        for (const auto& r : results) {
            std::cout << r.port << "\topen\t" << r.service;
            if (r.service.size() < 8) std::cout << "\t";
            std::cout << "\t" << r.banner << "\n";
        }
    }

    std::cout << "\nScanned " << (opts.endPort - opts.startPort + 1)

```

```
        << " ports in " << scanner.elapsed().count() << " ms. "
        << results.size() << " open.\n";

    return 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(tcp_port_scanner CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release)
endif()

add_compile_options(-Wall -Wextra)

find_package(Threads REQUIRED)

add_executable(scanner
    src/main.cpp
    src/scanner.cpp
)

target_include_directories(scanner PRIVATE include)
target_link_libraries(scanner PRIVATE Threads::Threads)
```

18 Where to Go Next

Concrete, increasingly challenging extensions, roughly in the order they're worth attempting:

Extension	What it teaches
IPv6 support	<code>AF_INET6</code> , <code>sockaddr_in6</code> , dual-stack resolution logic
<code>--no-banner</code> speed benchmarking	Measuring and reasoning about I/O-bound vs. CPU-bound performance
JSON output mode	Structuring a tool's output for other programs to consume, not just humans
CIDR range input (e.g. <code>192.168.1.0/24</code>)	Subnet math, scanning multiple hosts, aggregating results
Rate limiting	Token-bucket or leaky-bucket algorithms, being a "polite" network citizen
Raw-socket SYN scanning	Manual IP/TCP header construction, checksums, raw sockets, why root is required
UDP scanning	Handling the ambiguity of UDP — no handshake means "no reply" is not conclusive

Each of these is a self-contained project in the same spirit as this one: pick a real limitation, understand exactly why it exists, and build the smallest correct fix for it.